Viewfinder

A camera-focused ray tracer, built as a hidden-code course

Course notes, problem sets, tests, and verification for `github.com/cravies/raytracer`

Briefings, tests, and verification by Claude. Every line of raytracer code is Ben's.

September 2026

What this document is

This is the record of a learning experiment. Take *Ray Tracing in One Weekend*. Cut it down to the parts that teach camera models. Turn what remains into a course where the reader never sees the book's code. An AI (Claude) reads the book. It writes lectures that explain the concepts in full. It states each problem the way an exam would. It writes tests. The student (Ben) derives the mathematics on paper, implements everything in C++, and debugs it from rendered images. The stages already finished are documented here with the renders they produced. The stages still to come are stated as problems.

The book's authors write well. Where they have said the thing, this document uses their words. Each run of the book's prose ends in a footnote that gives the section. Claude's sentences do three jobs only: connect the book's paragraphs, add what the book leaves to the reader, and state the project's numbers and problems. Every equation and number here was checked by `verify_geometry.py`, shipped alongside. Appendix 13 says how.

Provenance and rules. Claude never writes raytracer code. It never reviews Ben's. It may write test scaffolding, which is marked as such in the repository. Briefings contain concepts and problem statements. They contain no implementation design: no data structures, no signatures, no step sequences. Conceptual questions get full answers at any time. "What's the next line" gets a hint at most. Bugs are Ben's to find, unaided. This document contains no debugging guidance. The rendered image is the feedback loop.

Priorities. Camera mathematics first. C++ second. 3D graphics third. That order explains what was cut from the book: materials, scattering, recursion, antialiasing, gamma, the `hittable` abstraction, intervals, shared pointers. Everything about what light does after it leaves the camera is gone. Everything about how it leaves is kept, plus one sphere to look at.

Status

Stage	Content (book section)	Status	Render
0	PPM writer, `vec3`, `ray` (§2–4.1)	done, Aug 31	—
1	Viewport geometry, sky gradient (§4.2)	done, Aug 31	Fig. 1
2	Flat red sphere (§5)	done, Sep 1	Fig. 2
3	Normals, ground sphere, grid scene (§6.1–6.3, no abstraction)	done, Sep 1	Figs. 3–4
4	Positionable camera (§12)	to do	—
5	Defocus blur (§13)	to do	—
6	Orbit render; rewrite scene builder unaided	to do	—

Stage 0 — Foundations (done)

1 The PPM writer, `vec3`, and `ray`

Whenever you start a renderer, you need a way to see an image. The most straightforward way is to write it to a file. The catch is, there are so many formats. Many of those are complex. I always start with a plain text ppm file. There are some things to note in this code: 1. The pixels are written out in rows. 2. Every row of pixels is written out left to right. 3. These rows are written out from top to bottom. 4. By convention, each of the red/green/blue components are represented internally by real-valued variables that range from 0.0 to 1.0. These must be scaled to integer values between 0 and 255 before we print them out. 5. Red goes from fully off (black) to fully on (bright red) from left to right, and green goes from fully off at the top (black) to fully on at the bottom (bright green). Adding red and green light together make yellow so we should expect the bottom right corner to be yellow.[1] The project writes the file through an `ofstream` instead of redirecting `cout`. Nothing else differs. That 256×256 gradient was the first image. It proved the pipeline before any geometry existed.

Almost all graphics programs have some class(es) for storing geometric vectors and colors. In many systems these vectors are 4D (3D position plus a homogeneous coordinate for geometry, or RGB plus an alpha transparency component for colors). For our purposes, three coordinates suffice. We'll use the same class `vec3` for colors, locations, directions, offsets, whatever. Some people don't like this because it doesn't prevent you from doing something silly, like subtracting a position from a color. They have a good point, but we're going to always take the "less code" route when not obviously wrong. In spite of this, we do declare two aliases for `vec3`: `point3` and `color`.[2] The project built `vec3` lazily. Each operator was added the first time a stage needed it, and each was pinned by an assert before use. The cross product is still missing. It is the first job of Stage 4.

The one thing that all ray tracers have is a ray class and a computation of what color is seen along a ray. Let's think of a ray as a function $\mathbf{P}(t) = \mathbf{A} + t\mathbf{b}$. Here $\mathbf{P}$ is a 3D position along a line in 3D. $\mathbf{A}$ is the ray origin and $\mathbf{b}$ is the ray direction. The ray parameter t is a real number (`double` in the code). Plug in a different t and $\mathbf{P}(t)$ moves the point along the ray. Add in negative t values and you can go anywhere on the 3D line. For positive t, you get only the parts in front of $\mathbf{A}$, and this is what is often called a half-line or a ray.[3] Two facts about t matter for everything that follows. First, t is measured in units of the direction vector, not in world distance. Directions are not normalized. Second, the sign of t is the only thing that separates "in front of the camera" from "behind it."

[1] RTiOW §2.1.
[2] RTiOW §3.
[3] RTiOW §4.1.

One trap in the notation, found the hard way. The book writes the ray as $\mathbf{P}(t) = \mathbf{Q} + t\mathbf{d}$ in the sphere chapter. In the camera chapter, $\mathbf{Q}$ is the viewport's upper-left corner. Two unrelated objects, one letter. The code keeps them apart.

Stage 1 — Viewport geometry and the sky (done)

2 Lecture: sending rays into the scene

Now we are ready to turn the corner and make a ray tracer. At its core, a ray tracer sends rays through pixels and computes the color seen in the direction of those rays. The involved steps are 1. Calculate the ray from the "eye" through the pixel, 2. Determine which objects the ray intersects, and 3. Compute a color for the closest intersection point. When first developing a ray tracer, I always do a simple camera for getting the code up and running.

I've often gotten into trouble using square images for debugging because I transpose x and y too often, so we'll use a non-square image. A square image has a 1:1 aspect ratio, because its width is the same as its height. Since we want a non-square image, we'll choose 16:9 because it's so common. The image's aspect ratio can be determined from the ratio of its width to its height. However, since we have a given aspect ratio in mind, it's easier to set the image's width and the aspect ratio, and then using this to calculate for its height. This way, we can scale up or down the image by changing the image width, and it won't throw off our desired aspect ratio. We do have to make sure that when we solve for the image height the resulting height is at least 1.

In addition to setting up the pixel dimensions for the rendered image, we also need to set up a virtual *viewport* through which to pass our scene rays. The viewport is a virtual rectangle in the 3D world that contains the grid of image pixel locations. If pixels are spaced the same distance horizontally as they are vertically, the viewport that bounds them will have the same aspect ratio as the rendered image. The distance between two adjacent pixels is called the pixel spacing, and square pixels is the standard. To start things off, we'll choose an arbitrary viewport height of 2.0, and scale the viewport width to give us the desired aspect ratio.

If you're wondering why we don't just use `aspect_ratio` when computing `viewport_width`, it's because the value set to `aspect_ratio` is the ideal ratio, it may not be the *actual* ratio between `image_width` and `image_height`. If `image_height` was allowed to be real valued—rather than just an integer—then it would be fine to use `aspect_ratio`. But the *actual* ratio between `image_width` and `image_height` can vary based on two parts of the code. First, `image_height` is rounded down to the nearest integer, which can increase the ratio. Second, we don't allow `image_height` to be less than one, which can also change the actual aspect ratio. Note that `aspect_ratio` is an ideal ratio, which we approximate as best as possible with the integer-based ratio of image width over image height. In order for our viewport proportions to exactly match our image proportions, we use the calculated image aspect ratio to determine our final viewport width.

Next we will define the camera center: a point in 3D space from which all scene rays will originate (this is also commonly referred to as the *eye point*). The vector from the camera center to the viewport center will be orthogonal to the viewport. We'll initially set the distance between the viewport and the camera center point to be one unit. This distance is often referred to as the *focal length*. For simplicity we'll start with the camera center at $(0, 0, 0)$. We'll also have the y-axis go up, the x-axis to the right, and the negative z-axis pointing in the viewing direction. (This is commonly referred to as *right-handed coordinates*.)

Now the inevitable tricky part. While our 3D space has the conventions above, this conflicts with our image coordinates, where we want to have the zeroth pixel in the top-left and work our way down to the last pixel at the bottom right. This means that our image coordinate Y-axis is

inverted: Y increases going down the image. As we scan our image, we will start at the upper left pixel (pixel $0, 0$), scan left-to-right across each row, and then scan row-by-row, top-to-bottom. To help navigate the pixel grid, we'll use a vector from the left edge to the right edge ($\mathbf{V_u}$), and a vector from the upper edge to the lower edge ($\mathbf{V_v}$). Our pixel grid will be inset from the viewport edges by half the pixel-to-pixel distance. This way, our viewport area is evenly divided into width $\times$ height identical regions. In this figure, we have the viewport, the pixel grid for a 7×5 resolution image, the viewport upper left corner $\mathbf{Q}$, the pixel $\mathbf{P_{0,0}}$ location, the viewport vector $\mathbf{V_u}$ (`viewport_u`), the viewport vector $\mathbf{V_v}$ (`viewport_v`), and the pixel delta vectors $\mathbf{\Delta u}$ and $\mathbf{\Delta v}$. Notice that in the code above, I didn't make `ray_direction` a unit vector, because I think not doing that makes for simpler and slightly faster code.[4]

Here are the project's numbers. Ben derived them by hand from those paragraphs and confirmed them with print statements before rendering a pixel. The camera sits at the origin. The gaze pierces the viewport at $(0, 0, -1)$.

$$\text{height} = 225, \quad \text{viewport width} = 3.55556, \quad \Delta u = (0.00888889, 0, 0), \quad \Delta v = (0, -0.00888889, 0),$$

$$\mathbf{Q} = (-1.77778,\ 1,\ -1), \qquad \mathbf{P}_{0,0} = (-1.77333,\ 0.995556,\ -1).$$

The project rounded the height up with `ceil`. The book truncates. At $400/(16/9) = 225$ exactly, the two agree.

Now we'll fill in the `ray_color(ray)` function to implement a simple gradient. This function will linearly blend white and blue depending on the height of the y coordinate *after* scaling the ray direction to unit length (so $-1.0 < y < 1.0$). Because we're looking at the y height after normalizing the vector, you'll notice a horizontal gradient to the color in addition to the vertical gradient. I'll use a standard graphics trick to linearly scale $0.0 \leq a \leq 1.0$. When $a = 1.0$, I want blue. When $a = 0.0$, I want white. In between, I want a blend. This forms a "linear blend", or "linear interpolation". This is commonly referred to as a *lerp* between two values. A lerp is always of the form $\mathit{blendedValue} = (1 - a) \cdot \mathit{startValue} + a \cdot \mathit{endValue}$, with a going from zero to one.[5]

The horizontal gradient the book mentions is the one non-obvious thing in this stage. It became the comprehension check. A ray direction's raw y mixes two things: how high the pixel sits, and how far away it is. A corner pixel is farther from the eye than the center pixel of its row. Its direction is longer. Dividing by length leaves a smaller unit y. Ben's account of it, at the end of day one: "Normalization divides by the whole ray's length. The ray is longer at the sides than the middle, norm-wise, even though the y value is the same."

Figure 1: Stage 1 render. Top-center pixel $(146, 190, 255)$. Top corners $(163, 200, 255)$. Center $(191, 217, 255)$. The verification script reproduces all three from the geometry.

[4] RTiOW §4.2, with the code listings and figures omitted.
[5] RTiOW §4.2.

Stage 2 — A sphere (done)

3 Lecture: ray-sphere intersection

Let's add a single object to our ray tracer. People often use spheres in ray tracers because calculating whether a ray hits a sphere is relatively simple.[6]

The equation for a sphere of radius r that is centered at the origin is an important mathematical equation: $x^2 + y^2 + z^2 = r^2$. You can also think of this as saying that if a given point (x, y, z) is on the surface of the sphere, then $x^2 + y^2 + z^2 = r^2$. If a given point (x, y, z) is *inside* the sphere, then $x^2 + y^2 + z^2 < r^2$, and if a given point (x, y, z) is *outside* the sphere, then $x^2 + y^2 + z^2 > r^2$. If we want to allow the sphere center to be at an arbitrary point (C_x, C_y, C_z), then the equation becomes a lot less nice: $(C_x - x)^2 + (C_y - y)^2 + (C_z - z)^2 = r^2$. In graphics, you almost always want your formulas to be in terms of vectors so that all the $x/y/z$ stuff can be simply represented using a `vec3` class. You might note that the vector from point $\mathbf{P} = (x, y, z)$ to center $\mathbf{C} = (C_x, C_y, C_z)$ is $(\mathbf{C} - \mathbf{P})$. If we use the definition of the dot product, then we can rewrite the equation of the sphere in vector form as

$$(\mathbf{C} - \mathbf{P}) \cdot (\mathbf{C} - \mathbf{P}) = r^2.$$

We can read this as "any point $\mathbf{P}$ that satisfies this equation is on the sphere". We want to know if our ray $\mathbf{P}(t) = \mathbf{Q} + t\mathbf{d}$ ever hits the sphere anywhere. If it does hit the sphere, there is some t for which $\mathbf{P}(t)$ satisfies the sphere equation. So we are looking for any t where this is true: $(\mathbf{C} - \mathbf{P}(t)) \cdot (\mathbf{C} - \mathbf{P}(t)) = r^2$, which can be found by replacing $\mathbf{P}(t)$ with its expanded form:

$$(\mathbf{C} - (\mathbf{Q} + t\mathbf{d})) \cdot (\mathbf{C} - (\mathbf{Q} + t\mathbf{d})) = r^2.$$

We have three vectors on the left dotted by three vectors on the right. If we solved for the full dot product we would get nine vectors. You can definitely go through and write everything out, but we don't need to work that hard. If you remember, we want to solve for t, so we'll separate the terms based on whether there is a t or not:

$$(-t\mathbf{d} + (\mathbf{C} - \mathbf{Q})) \cdot (-t\mathbf{d} + (\mathbf{C} - \mathbf{Q})) = r^2.$$

And now we follow the rules of vector algebra to distribute the dot product, and move the square of the radius over to the left hand side:

$$t^2\, \mathbf{d} \cdot \mathbf{d} - 2t\, \mathbf{d} \cdot (\mathbf{C} - \mathbf{Q}) + (\mathbf{C} - \mathbf{Q}) \cdot (\mathbf{C} - \mathbf{Q}) - r^2 = 0.$$

It's hard to make out what exactly this equation is, but the vectors and r in that equation are all constant and known. Furthermore, the only vectors that we have are reduced to scalars by dot product. The only unknown is t, and we have a t^2, which means that this equation is quadratic. You can solve for a quadratic equation $ax^2 + bx + c = 0$ by using the quadratic formula $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$. So solving for t in the ray-sphere intersection equation gives us these values for a, b, and c:

$$a = \mathbf{d} \cdot \mathbf{d}, \qquad b = -2\, \mathbf{d} \cdot (\mathbf{C} - \mathbf{Q}), \qquad c = (\mathbf{C} - \mathbf{Q}) \cdot (\mathbf{C} - \mathbf{Q}) - r^2.$$

Using all of the above you can solve for t, but there is a square root part that can be either positive (meaning two real solutions), negative (meaning no real solutions), or zero (meaning one real solution). In graphics, the algebra almost always relates very directly to the geometry.[7]

[6] RTiOW §5.

[7] RTiOW §5.1.

The book leaves that last sentence as an assertion. Appendix 13 makes it exact. Let $h = \mathbf{d} \cdot (\mathbf{C} - \mathbf{Q})$ and let δ be the perpendicular distance from the center to the ray's line. Then $h^2 - ac = |\mathbf{d}|^2 (r^2 - \delta^2)$. So the discriminant is non-negative exactly when the line passes within r of the center. Under the course's rules this derivation was given, as in a physics class. The problem was implementing it. Ben did so from the coefficient formulas alone.

If we take that math and hard-code it into our program, we can test our code by placing a small sphere at -1 on the z-axis and then coloring red any pixel that intersects it. Now this lacks all sorts of things—like shading, reflection rays, and more than one object—but we are closer to halfway done than we are to our start! One thing to be aware of is that we are testing to see if a ray intersects with the sphere by solving the quadratic equation and seeing if a solution exists, but solutions with negative values of t work just fine. If you change your sphere center to $z = +1$ you will get exactly the same picture because this solution doesn't distinguish between objects *in front of the camera* and objects *behind the camera*. This is not a feature![8]

A number worth knowing. The disc measured 130 pixels across. The naive estimate is 112: sphere diameter 1.0 against viewport width 3.556, over 400 pixels. The difference is not an error. The silhouette is made by tangent rays. Their half-angle is $\arcsin(r/\text{dist})$. The projected radius on the viewport plane is therefore $\tan(\arcsin 0.5) = 0.577$, not 0.5. That is 65 pixels of radius and 130 across, as measured. A sphere at finite distance always looks a little bigger than its cross-section.

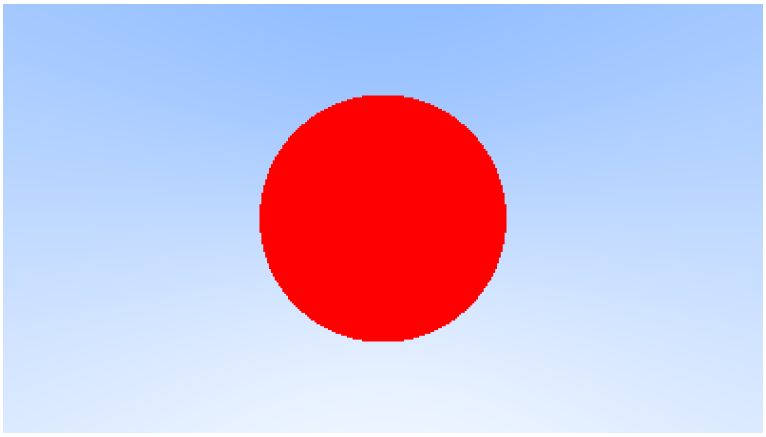

Figure 2: Stage 2 render. One flat-color sphere at $(0, 0, -1)$, radius 0.5, 130 pixels across.

Stage 3 — Normals, a ground, and a field of spheres (done)

4 Lecture: shading with surface normals

First, let's get ourselves a surface normal so we can shade. This is a vector that is perpendicular to the surface at the point of intersection. We have a key design decision to make for normal vectors in our code: whether normal vectors will have an arbitrary length, or will be normalized to unit length. It is tempting to skip the expensive square root operation involved in normalizing the vector, in case it's not needed. In practice, however, there are three important observations. First, if a unit-length normal vector is *ever* required, then you might as well do it up front once, instead

[8] RTiOW §5.2.

of over and over again "just in case" for every location where unit-length is required. Second, we *do* require unit-length normal vectors in several places. Third, if you require normal vectors to be unit length, then you can often efficiently generate that vector with an understanding of the specific geometry class, in its constructor, or in the `hit()` function. For example, sphere normals can be made unit length simply by dividing by the sphere radius, avoiding the square root entirely. Given all of this, we will adopt the policy that all normal vectors will be of unit length.

For a sphere, the outward normal is in the direction of the hit point minus the center. On the earth, this means that the vector from the earth's center to you points straight up. Let's throw that into the code now, and shade it. We don't have any lights or anything yet, so let's just visualize the normals with a color map. A common trick used for visualizing normals (because it's easy and somewhat intuitive to assume $\mathbf{n}$ is a unit length vector—so each component is between -1 and 1) is to map each component to the interval from 0 to 1, and then map (x, y, z) to $(red, green, blue)$. For the normal, we need the hit point, not just whether we hit or not (which is all we're calculating at the moment). We only have one sphere in the scene, and it's directly in front of the camera, so we won't worry about negative values of t yet. We'll just assume the closest hit point (smallest t) is the one that we want.[9] The center of the disc came out $(128, 127, 255)$. The normal $(0, 0, 1)$ maps to $(0.5, 0.5, 1.0)$. That is what the geometry demands.

First, recall that a vector dotted with itself is equal to the squared length of that vector. Second, notice how the equation for `b` has a factor of negative two in it. Consider what happens to the quadratic equation if $b = -2h$:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a} = \frac{-(-2h) \pm \sqrt{(-2h)^2 - 4ac}}{2a} = \frac{2h \pm 2\sqrt{h^2 - ac}}{2a} = \frac{h \pm \sqrt{h^2 - ac}}{a}.$$

This simplifies nicely, so we'll use it. So solving for h: $b = -2\,\mathbf{d} \cdot (\mathbf{C} - \mathbf{Q})$, $b = -2h$, $h = \frac{b}{-2} = \mathbf{d} \cdot (\mathbf{C} - \mathbf{Q})$.[10]

5 Lecture: many spheres

Now, how about more than one sphere? While it is tempting to have an array of spheres, a very clean solution is to make an "abstract class" for anything a ray might hit, and make both a sphere and a list of spheres just something that can be hit. What that class should be called is something of a quandary—calling it an "object" would be good if not for "object oriented" programming. "Surface" is often used, with the weakness being maybe we will want volumes (fog, clouds, stuff like that). "hittable" emphasizes the member function that unites them. I don't love any of these, but we'll go with "hittable". This `hittable` abstract class will have a `hit` function that takes in a ray. Most ray tracers have found it convenient to add a valid interval for hits t_{min} to t_{max}, so the hit only "counts" if $t_{min} < t < t_{max}$. For the initial rays this is positive t, but as we will see, it can simplify our code to have an interval t_{min} to t_{max}. One design question is whether to do things like compute the normal if we hit something. We might end up hitting something closer as we do our search, and we will only need the normal of the closest thing.[11]

The project took the tempting road on purpose. A sphere is a plain struct of center and radius. The scene is a vector of them. The search is a loop. The abstraction exists to hold different kinds of objects. This project has one kind. So the array is what `hittable` collapses to. It also matches the C-style discipline Ben is practicing: data in structs, behavior in free functions.

Two conditions from the book's design survive. The hit must be the closest one found during the search. It counts only for positive t. Section 5.2 warned that the quadratic will otherwise report objects behind the camera. Once t values are compared across objects, a bogus negative

[9] RTiOW §6.1.
[10] RTiOW §6.2.
[11] RTiOW §6.3.

root can beat every real hit. The search must also report which sphere won. The normal needs that sphere's center.

The second sphere is the book's own: a ground sphere with center $(0, -100.5, -1)$ and radius 100. This yields a picture that is really just a visualization of where the spheres are located along with their surface normal. This is often a great way to view any flaws or specific characteristics of a geometric model.[12] The floor came out $(128, 255, 128)$, from the upward normal $(0, 1, 0)$.

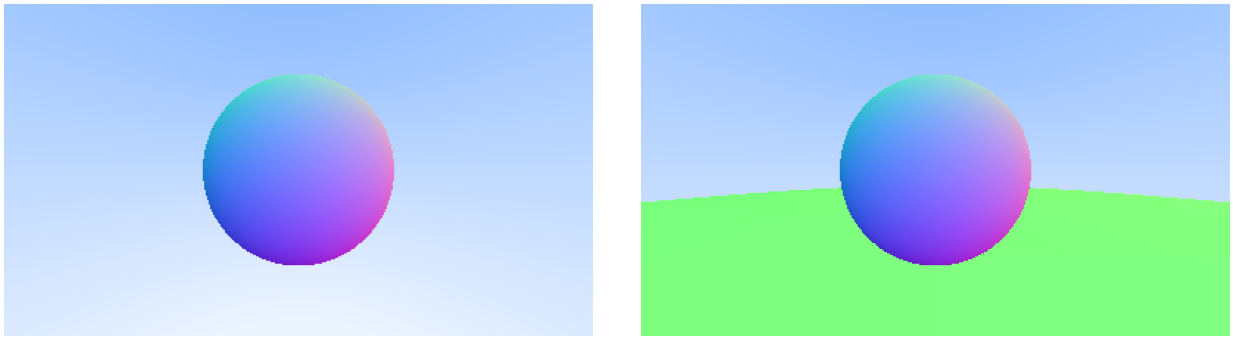

Figure 3: Left: Stage 3a, normal-shaded sphere. Center pixel $(128, 127, 255)$. Right: Stage 3b, ball on the ground sphere. Floor $(128, 255, 128)$. The occlusion boundary is clean.

6 The grid of spheres, and a lesson about eye level

The grid was added to the plan. A regular pattern turns camera bugs into something you can see. Perspective convergence, roll, and field-of-view distortion all show on a grid, the way they show on a calibration checkerboard. The grid code is pure scene construction: two nested loops placing small spheres. Asserts check the count, the spacing, the shared radius, and that the balls rest on the ground.

The first render was a band of overlapping beads along the horizon. It was not a field receding into the distance. The cause was geometry, not a bug. The ball centers sat at $y = 0$, the camera's own height. Every sphere projected onto the horizon row. A tabletop viewed from table height is a line. The verification script reproduces this: a ball at camera height projects to $y = 0$ on the viewport, whatever its x or depth. Lowering the scene made the layout visible. It is still not a bird's-eye view. The gaze is horizontal. The difference between "table below the sightline" and "looking down at the table" is what Stage 4 delivers.

A second observation from this render, asked and answered. Spheres near the frame's edge look wider than those dead ahead. That is flat-image-plane perspective. A sphere subtends the same angle from the eye wherever it sits. But an oblique slice through its cone of rays is stretched along the direction of tilt, by about $1/\cos$ of the off-axis angle. Real pinhole cameras do the same at wide angles. Lens distortion is a separate effect on top. Appendix 13 measures the stretch of a projected silhouette and matches it to $1/\cos\phi$.

[12] RTiOW §6.6.

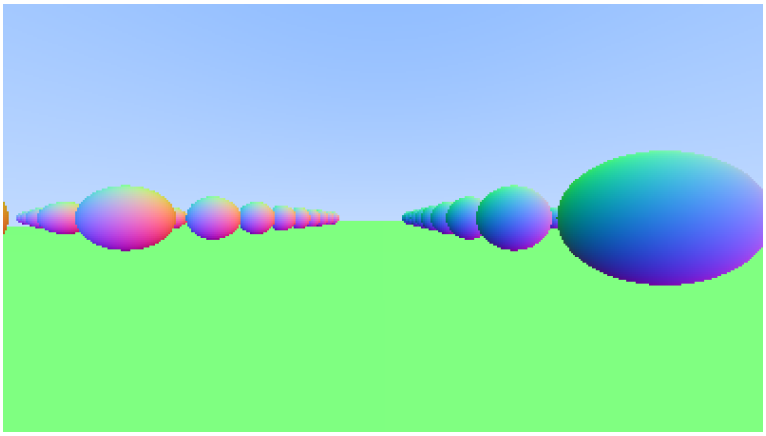

Figure 4: Stage 3c, the first grid render. Eye-level placement collapses the grid onto the horizon. The scene was then lowered. This render is kept as the "before" picture for Stage 4.

Stage 4 — The positionable camera (to do)

7 Lecture

Cameras, like dielectrics, are a pain to debug, so I always develop mine incrementally.[13] The book's increments are the course's. First an adjustable field of view, with the camera still fixed. Then position and orientation. Everything built so far assumes three facts silently: the eye at the origin, the gaze along $-z$, up meaning $+y$. Those are not camera properties. They are a choice of coordinates, baked into every line of the viewport setup. A positionable camera means removing them while keeping the construction.

7.1 Field of view

First, let's allow for an adjustable field of view (*fov*). This is the visual angle from edge to edge of the rendered image. Since our image is not square, the fov is different horizontally and vertically. I always use vertical fov. I also usually specify it in degrees and change to radians inside a constructor—a matter of personal taste. First, we'll keep the rays coming from the origin and heading to the $z = -1$ plane. We could make it the $z = -2$ plane, or whatever, as long as we made h a ratio to that distance. This implies $h = \tan(\frac{\theta}{2})$.[14] The book's figure is one right triangle. It is omitted here, so in words: the viewport hangs a focal length f ahead of the eye. Its top edge sits half the viewport height above the gaze line. The angle at the eye is $\theta/2$. The adjacent side is f. The opposite side is the half-height h. So $h = f\tan(\theta/2)$, and the viewport height is $2h$. The book has $f = 1$. Width follows from the aspect ratio as before.

A check you can do in your head. At $\theta = 90°$ and $f = 1$, $\tan 45° = 1$. The viewport height is exactly 2.0, the "arbitrary" value of §4.2. So the fov camera reproduces everything already rendered when set to 90°. A wide angle means a bigger viewport, more world in frame, everything smaller. A narrow angle means zoom. Zoom scales with the ratio of tangents, not of angles. Halving the viewport height takes the fov from 90° to $2\arctan(0.5) \approx 53.1°$.

7.2 Position and orientation

To get an arbitrary viewpoint, let's first name the points we care about. We'll call the position where we place the camera *lookfrom*, and the point we look at *lookat*. (Later, if you want, you could define a direction to look in instead of a point to look at.) We also need a way to specify

[13] RTiOW §12.
[14] RTiOW §12, §12.1.

the roll, or sideways tilt, of the camera: the rotation around the lookat-lookfrom axis. Another way to think about it is that even if you keep `lookfrom` and `lookat` constant, you can still rotate your head around your nose. What we need is a way to specify an "up" vector for the camera.[15]

We can specify any up vector we want, as long as it's not parallel to the view direction. Project this up vector onto the plane orthogonal to the view direction to get a camera-relative up vector. I use the common convention of naming this the "view up" (*vup*) vector. After a few cross products and vector normalizations, we now have a complete orthonormal basis (u, v, w) to describe our camera's orientation. u will be the unit vector pointing to camera right, v is the unit vector pointing to camera up, w is the unit vector pointing opposite the view direction (since we use right-hand coordinates), and the camera center is at the origin.[16] Like before, when our fixed camera faced $-Z$, our arbitrary view camera faces $-w$. Keep in mind that we can—but we don't have to—use world up $(0, 1, 0)$ to specify vup. This is convenient and will naturally keep your camera horizontally level until you decide to experiment with crazy camera angles.[17]

That paragraph is the whole exercise. Here is the reasoning behind "a few cross products." A camera, wherever it sits, has three natural directions: the way it looks, what is right in its view, and what is up in its view. Make them unit length and mutually perpendicular. That is an orthonormal basis, the camera's private frame. Your current camera is the special case $u = (1, 0, 0)$, $v = (0, 1, 0)$, $w = (0, 0, 1)$.

The construction has three steps. First, w: the unit vector from lookat back toward lookfrom. It points backward along the gaze. Second, u: you need a "right" that is perpendicular to both the gaze and the up hint. Making a vector perpendicular to two others is what the cross product does. Cross vup with w and normalize. Third, v: cross w with u. It is perpendicular to both by construction. It is already unit length, because w and u are unit and perpendicular. Lagrange's identity, $|w \times u|^2 = |w|^2|u|^2 - (w \cdot u)^2$, makes that exact. And it is the true up: vup with its along-gaze part removed.

Order matters in both crosses. The cross product is anticommutative. Swap the arguments and the result flips. A flipped basis vector renders as a mirrored image or an upside-down camera. This is Gram–Schmidt's rough cousin. The up hint is not perpendicular to the gaze, and the crosses remove the part that is not. It is also, literally, the rotation R in the $K[R\,|\,t]$ camera matrices your team's C++ uses.

The cross product itself, since `vec3` lacks it. For $\mathbf{a} = (a_1, a_2, a_3)$ and $\mathbf{b} = (b_1, b_2, b_3)$,

$$\mathbf{a} \times \mathbf{b} = (a_2b_3 - a_3b_2,\ a_3b_1 - a_1b_3,\ a_1b_2 - a_2b_1).$$

It is perpendicular to both inputs. It follows the right-hand rule. Its magnitude is $|\mathbf{a}||\mathbf{b}| \sin\angle$. And $\mathbf{b} \times \mathbf{a} = -\mathbf{a} \times \mathbf{b}$.

7.3 What the book's paragraphs assume

The book's camera code does several things without its prose saying why. You will not see the code. So they are stated here.

Why w points backward. The book says w is opposite the view direction "since we use right-hand coordinates," and stops. Here is the reason. Put u to the right and v up. A right-handed triple (u, v, w) forces $w = u \times v$ to point toward the viewer, out of the image. If you made the third axis the gaze instead, (u, v, gaze) would be left-handed. Every cross-product identity you rely on would pick up a sign. So the camera looks along $-w$. The fixed camera looks along $-z$ with $w = +z$, the same way. It is a convention. Once chosen, it is load-bearing in every formula.

[15] RTiOW §12.2.
[16] RTiOW §12.2.
[17] RTiOW §12.2.

vup is a hint, not a vector you keep. Two up hints that differ by a multiple of the gaze produce the same frame. The cross with w kills the along-gaze component. That is what "project this up vector onto the plane orthogonal to the view direction" means in practice. You never compute the projection. The cross products do it. So vup fixes only the roll. With world up $(0, 1, 0)$ the camera stays level. The one input the construction cannot take is a vup parallel to the gaze. Then $\text{cross}(\text{vup}, w) = 0$ and u is undefined. Looking straight down needs a different hint.

An orthonormal frame is a change of coordinates. Stack u, v, w as the rows of a 3×3 matrix R. A world vector's coordinates in the camera's frame are its dot products with u, v, w. That is the product $R\mathbf{x}$. Because the rows are orthonormal, $R^{\mathsf{T}}R = I$. The inverse of R is its transpose. Going from camera coordinates back to world is R^{T}. The pixel grid you build is this in disguise. A pixel offset $(x_{\text{off}}, y_{\text{off}})$ on a viewport at focal length f is the camera-coordinate direction $(x_{\text{off}}, y_{\text{off}}, -f)$. Its world direction is $x_{\text{off}}\,u + y_{\text{off}}\,v - f\,w = R^{\mathsf{T}}(x_{\text{off}}, y_{\text{off}}, -f)$. That is the whole content of "extrinsics" in the computer-vision sense. Appendix 13 checks it.

Where the viewport sits does not matter. For a pinhole camera, only ray directions determine the image. Scale the focal length by k and the viewport scales by k. Every direction stays the same. So the book's choice of focal length, the lookfrom–lookat distance, is not a geometric necessity. It is a convenience. It puts the viewport plane through lookat, so the center pixel lands on the point you named. It also sets up Stage 5, which reuses the same plane as the focus plane. Verified in the appendix at focal lengths from $0.5\times$ to $10\times$.

Degrees and radians. The book's helper converts because `std::tan` takes radians. Two traps are easy to walk into and are yours to avoid. The tangent is of $\theta/2$, not θ. And `M_PI` is not standard C++ on every compiler, so define π yourself as the book does.

What the render should show, geometrically. A pinhole maps straight lines to straight lines. The grid's rows stay straight under any camera. Lines that are parallel in the world converge in the image to a vanishing point. For lines in the ground plane, that point sits on the horizon. The horizon is the image of the ground plane's points at infinity. For a level camera it is the row where ray directions have zero world-y: the middle row of the frame. Pitch the camera down and it rises toward the top. These are the checks a grid render gives you. None of them needs the reference image.

7.4 Rebuilding the viewport in the new frame

Every step of the existing setup survives with its axes swapped. The viewport's horizontal span runs along u. Its vertical span runs along $-v$, down the image, as your down-vector runs along $-y$ today. The walk from the eye to the viewport center goes a focal length along $-w$ instead of along $-z$. The eye is lookfrom, not the origin. Corner, half-pixel inset, per-pixel deltas: the same logic with new ingredients. Your ray generation already subtracts the camera position from the pixel position. That line was written on day one "for when the camera moves." It is ready. One convention the book adopts: the focal length is the distance from lookfrom to lookat. The viewport plane then passes through lookat, and the pixel at the exact center of the image sits at lookat itself.

8 Problem 4

Build. The cross product first, with asserts. Then the frame (u, v, w) from lookfrom, lookat, and vup. Then the viewport: height from vertical fov and focal length, width from aspect ratio,

spans along u and $-v$, corner and anchor and deltas as before but in the new basis, camera center at lookfrom. Then render your grid scene from a new viewpoint.

Givens. Image 400×225. First render: lookfrom $(-2, 2, 1)$, lookat the center of your grid, vup $(0, 1, 0)$, vertical fov $90°$. Second render: the same with fov $20°$. Third render: lookfrom at the origin, lookat $(0, 0, -1)$, fov $90°$. Confirm this one is pixel-identical to your Stage 3 render.

Acceptance. The grid seen from above and beside. Rows converge with perspective. Balls stay circular. The horizon sits where the geometry puts it. The $20°$ render is a zoom of the $90°$ one: the grid grows, nothing distorts. The third render matches Stage 3 exactly. The tests in Appendix 14 check the frame's orthonormality and handedness, the fov formula at $90°$, the viewport center landing on lookat, and the general camera reproducing the day-one numbers.

Stage 5 — Defocus blur (to do)

9 Lecture

Now our final feature: *defocus blur*. Note, photographers call this *depth of field*, so be sure to only use the term *defocus blur* among your raytracing friends. The reason we have defocus blur in real cameras is because they need a big hole (rather than just a pinhole) through which to gather light. A large hole would defocus everything, but if we stick a lens in front of the film/sensor, there will be a certain distance at which everything is in focus. Objects placed at that distance will appear in focus and will linearly appear blurrier the further they are from that distance. You can think of a lens this way: all light rays coming *from* a specific point at the focus distance—and that hit the lens—will be bent back *to* a single point on the image sensor.[18]

We call the distance between the camera center and the plane where everything is in perfect focus the *focus distance*. Be aware that the focus distance is not usually the same as the focal length—the *focal length* is the distance between the camera center and the image plane. For our model, however, these two will have the same value, as we will put our pixel grid right on the focus plane, which is *focus distance* away from the camera center.[19] For our virtual camera, we can have a perfect sensor and never need more light, so we only use an aperture when we want defocus blur.[20]

9.1 The thin lens

A real camera has a complicated compound lens. For our code, we could simulate the order: sensor, then lens, then aperture. Then we could figure out where to send the rays, and flip the image after it's computed (the image is projected upside down on the film). Graphics people, however, usually use a thin lens approximation. We don't need to simulate any of the inside of the camera—for the purposes of rendering an image outside the camera, that would be unnecessary complexity. Instead, I usually start rays from an infinitely thin circular "lens", and send them toward the pixel of interest on the focus plane (`focal_length` away from the lens), where everything on that plane in the 3D world is in perfect focus. In practice, we accomplish this by placing the viewport in this plane. Putting everything together: 1. The focus plane is orthogonal to the camera view direction. 2. The focus distance is the distance between the camera center and the focus plane. 3. The viewport lies on the focus plane, centered on the camera view direction vector. 4. The grid

[18] RTiOW §13.
[19] RTiOW §13.
[20] RTiOW §13.

of pixel locations lies inside the viewport (located in the 3D world). 5. Random image sample locations are chosen from the region around the current pixel location. 6. The camera fires rays from random points on the lens through the current image sample location.[21]

Here is why that produces blur, in one triangle the book leaves implicit. A ray leaves the lens at transverse offset s from the camera center. It passes through a point on the focus plane at distance f. At any depth D along the same line of sight, its transverse position is $s(1 - D/f)$. At $D = f$ this is zero for every s. All lens points send their ray through the same focus-plane point, and a scene point there renders sharp. At $D = 2f$ the offset is $-s$. The bundle has spread to the full lens radius. Averaging the rays smears whatever is there over a disc that size. Blur radius grows linearly with $|D - f|$. That is the book's "linearly appear blurrier." Appendix 13 checks it symbolically and numerically.

9.2 Generating sample rays

Without defocus blur, all scene rays originate from the camera center (or `lookfrom`). In order to accomplish defocus blur, we construct a disk centered at the camera center. The larger the radius, the greater the defocus blur. You can think of our original camera as having a defocus disk of radius zero (no blur at all), so all rays originated at the disk center (`lookfrom`). So, how large should the defocus disk be? Since the size of this disk controls how much defocus blur we get, that should be a parameter of the camera class. We could just take the radius of the disk as a camera parameter, but the blur would vary depending on the projection distance. A slightly easier parameter is to specify the angle of the cone with apex at viewport center and base (defocus disk) at the camera center. This should give you more consistent results as you vary the focus distance for a given shot. Since we'll be choosing random points from the defocus disk, we'll need a function to do that: `random_in_unit_disk()`. This function works using the same kind of method we use in `random_unit_vector()`, just for two dimensions.[22] That method is rejection. Draw a point in the $[-1, 1]^2$ square. Keep it if it lies inside the unit circle. Otherwise draw again. The acceptance rate is $\pi/4$. The disk lies in the camera's u–v plane, so it moves with the camera. Its radius follows from the cone angle by the same right triangle as the field of view: focus distance times tan(angle/2). A 10° defocus angle at focus distance 3.4 gives a radius of about 0.2975. Scale the accepted point by the disk's u and v radii and add it to the camera center. That is a ray origin. The direction is the pixel position minus that origin, as before.

9.3 What the book's paragraphs assume

Focus distance is a separate parameter. In the book's final camera, the viewport sits at `focus_dist`. That is a parameter independent of the lookfrom–lookat distance. The book's example puts lookat at one distance and `focus_dist` at 3.4. This follows from pinhole invariance. With zero defocus angle, moving the viewport plane changes nothing. So you are free to put it where the sharp plane should be. Defocus is what makes the plane's position visible. Keep the two ideas apart: lookat fixes direction. Focus distance fixes which depth is sharp.

Item 5 of the thin-lens list is optional here. "Random image sample locations are chosen from the region around the current pixel location" is the antialiasing jitter this course cut. Firing through the pixel center is fine. The defocus disk provides the randomness that matters for blur.

Random numbers in C++. The book uses `<random>`: an engine (`std::mt19937`) and a `uniform_real_distribution`. Two facts about them the book's code embodies without comment. Building a fresh engine on every call is slow, and it is statistically poor. One engine, made once

[21] RTiOW §13.1.
[22] RTiOW §13.2.

and reused, is the intent. Seeding it with a fixed value makes renders reproducible. You will want that when comparing a before and an after.

The disk is in the camera's plane. The two disk basis vectors are u and v scaled by the disk radius. They are not world x and y. When the camera moves, the lens moves with it. A ray's origin is the camera center plus a random point on that disk. Its direction is still pixel position minus origin. The origin now varies per ray.

9.4 An honest note on sampling

True defocus blur is an average over many rays per pixel, each from a different lens point. Averaging many rays per pixel is the antialiasing machinery this course cut. With one sample per pixel, each pixel gets one random lens point. Out-of-focus regions come out grainy rather than smoothly blurred. But the blur is real and its radius is right. You have two honest options. Accept the grain, and observe that the focused plane is sharp while everything else speckles. Or pull in the minimal multi-sample loop: sum N ray colors and divide by N. Decide when you see the first render.

10 Problem 5

Build. A defocus angle parameter and the disk it defines in the camera's u–v plane. Unit-disk rejection sampling. Ray origins drawn from the disk when the angle is positive, from the camera center otherwise. The viewport placed at the focus distance. For this camera that equals the lookfrom–lookat distance already in use.

Givens. Your grid scene. lookfrom $(-2, 2, 1)$, lookat the grid center, fov $20°$, defocus angle $10°$, focus distance equal to the lookfrom–lookat distance. Then a second render with the focus distance set shorter than that. Then a third with it longer.

Acceptance. Spheres at the focus distance render sharp. Spheres nearer or farther blur, more so the farther they are from the focus plane. Changing the focus distance moves the sharp band through the grid. Setting the defocus angle to zero reproduces the Stage 4 render exactly. Grain in the blurred regions is expected at one sample per pixel. The tests check the disk radius formula, that every disk sample lies inside the disk and in the u–v plane, and that a zero angle gives zero-length disk vectors.

Stage 6 — The payoff (to do)

11 The orbit render

The final artifact is a sequence of frames with the camera orbiting the grid. lookfrom moves on a circle around a fixed lookat, at a fixed height. One PPM per frame, stitched to a GIF or MP4 afterward. The parametrization is the one from any physics course. For angle ϕ stepping around the circle, lookfrom = lookat + $(\rho\cos\phi,\ h,\ \rho\sin\phi)$, with orbit radius ρ and height h. lookat and vup stay fixed. Name the frames with zero-padded indices so they sort. The stitching is build tooling, squarely in the delegate-to-AI category. `ffmpeg` reads a numbered PPM sequence directly: `ffmpeg -framerate 24 -i frame_%03d.ppm orbit.mp4`. It can emit a GIF the same way.

Each frame is the same camera called with a new lookfrom. Nothing new is implemented. The verification script confirms that for every angle on such an orbit, the viewport center lands exactly on lookat. The target stays centered while the world turns around it. That one image proves lookfrom, lookat, the frame, and the field of view all work. Add defocus with the focus distance on the grid center, and the orbit also shows depth of field. Put it at the top of the README.

12 Rewrite the scene builder, unaided

On September 1st, out of time at 5pm, Ben had Claude diagnose one bug in the grid-construction code. An offset had been written when the sphere radius was 0.03. It became a large shift when the radius changed. The commit message records this. The stated resolution is to rewrite that function from scratch, without reference, at the end of the project. That is the right retention check regardless of the accounting. If the rewrite takes ten minutes, the week worked. After it, the README's claim becomes exact: every line of raytracer code written and debugged by Ben, and Claude's code confined to tests, clearly marked.

What this maps to in computer vision

Everything built here is the pinhole camera model, run backward. Computer vision runs it forward, from world point to pixel, as $K[R\,|\,t]$. The correspondence is exact. The frame (u, v, w) from lookfrom, lookat, and vup is R. lookfrom is the translation. Together they are the extrinsics. The viewport construction is the intrinsics K: focal length, field of view, pixel steps, and the half-pixel offset to the first pixel center. In K these appear as focal length in pixels, the principal point, and the mapping between pixel indices and directions in space. Triangulation, which motivated this project, is the inverse of what the render loop does. Two cameras each run the pixel-to-ray conversion built in Stages 1 and 4. A least-squares step intersects the two rays. The learnopengl.com Getting Started chapters, which the team lead assigned alongside this book, express the same camera as 4×4 homogeneous matrices. After this project, those chapters should read as notation for a machine already built by hand.

---

13 Appendix: verification

Every equation and number above was checked by `verify_geometry.py`, shipped alongside this document. It needs only `numpy` and `sympy`. Forty-five checks, of three kinds.

Symbolic. The ray-sphere substitution expands to exactly the book's a, b, c. The h-form root equals the standard-formula root, and $b^2 - 4ac = 4(h^2 - ac)$. The discriminant's geometric meaning: $h^2 - ac = |\mathbf{d}|^2(r^2 - \delta^2)$, with δ the center-to-line distance. The sphere normal $(\mathbf{P} - \mathbf{C})/r$ is unit length for every surface point. Viewport height at 90° and $f = 1$ is exactly 2. For the frame: Lagrange's identity, so v is unit; $v \perp u$ and $v \perp w$ identically; $u \times v = w(u \cdot u) - u(u \cdot w)$, so the frame is right-handed once u and w are orthonormal. The defocus offset at depth D is $s(1 - D/f)$, zero at the focus plane.

Numeric, the project's own numbers. Height 225, where rounding up and truncation agree. Viewport width 3.55556. Both pixel steps 0.00888889. Corner $(-1.77778, 1, -1)$. Anchor $(-1.77333, 0.995556, -1)$. The corner bow: unit y of 0.7055 at top-center against 0.4393 at the top corner, from equal raw y. The resulting red values 146 and 163 match the render. The

disc: $\tan(\arcsin 0.5) = 0.5774$, 65 pixels of radius, 130 across, matching the measurement. The disc-center normal color and the floor color. Eye-level placement projecting to the horizon row. Off-axis sphere stretch 1.567 against $1/\cos\phi = 1.562$. The frame at lookfrom $(-2, 2, 1)$: orthonormal, right-handed, gaze $-w$, level, and its mirror under swapped cross arguments. The general camera at the original pose reproduces the day-one anchor and deltas exactly. Zoom as a ratio of tangents. Blur radius 10^{-17} at the focus plane and equal to the disk radius at twice it. Defocus radius 0.2975 for 10° at 3.4. Pinhole invariance: focal lengths from 0.5× to 10× leave every ray direction unchanged. The change of basis: $R\mathbf{x}$ equals the three dot products, $R^{\mathsf{T}}R = I$, and the pixel direction $x\,u + y\,v - f\,w$ equals $R^{\mathsf{T}}(x, y, -f)$. vup roll invariance: adding any multiple of the gaze to vup leaves the frame unchanged. Horizon: a level camera puts it on the middle row; pitching down raises it.

Fuzzing. Five hundred random six-sphere scenes: the closest positive root lands on the winning sphere's surface, and nothing is entered before it. Five hundred spheres behind the camera yield only negative roots. A thousand random normals: the remap stays in $[0, 1]^3$ and inverts exactly. A thousand random cameras: orthonormal, right-handed, $-w$ is the gaze, v aligns with vup. The one degenerate input, vup parallel to the gaze. Disk rejection sampling: acceptance 0.7851 against $\pi/4 = 0.7854$, every accepted point inside. Five hundred defocus rays: origin within the disk radius and in the u–v plane, reaching the focus-plane pixel at $t = 1$. Sixty orbit frames: viewport center on lookat at every angle.

The C++ tests. `tests_camera.cpp` was compiled with `g++ -std=c++20` against a private reference implementation of the interface contract. That reference was built on the project's own `vec3.h`, `ray.h`, and `sphere.h`. All tests pass there. The reference is not shipped. The tests will not pass against the repository as of September 2nd. That is expected. They target stages not yet built, and the moved-camera hit tests exercise a code path the existing hit function has never run.

14 Appendix: the test file

Add the file to the executable's sources in the CMake list. Declare `camera_tests()` where the existing `tests()` can see it, and call it from there. The header comment states the interface contract: a handful of names and shapes the tests need in order to call your code. Nothing behind those names is constrained.

```
//
// tests_camera.cpp -- tests for the positionable camera and defocus blur stages.
// Written by Claude (test scaffolding only). Everything they call is yours.
//
// Interface contract (the ONLY thing these tests assume). Everything behind it is your design:
//
//   vec3 cross(const vec3& u, const vec3& v);                      // in vec3.h
//
//   struct camera_frame { vec3 u, v, w; };
//   camera_frame make_frame(const vec3& lookfrom, const vec3& lookat, const vec3& vup);
//
//   double viewport_height_from_fov(double vfov_degrees, double focal_length);
//
//   struct viewport {                                              // everything the render
    loop needs
//       vec3 pixel00_loc, pixel_delta_u, pixel_delta_v;            // pixel grid
//       vec3 center;                                               // ray origin when there is
    no blur
//       vec3 defocus_disk_u, defocus_disk_v;                       // zero vectors when
    defocus_angle == 0
//   };
```

```
//   viewport make_viewport(const vec3& lookfrom, const vec3& lookat, const vec3& vup,
//                          double vfov_degrees, double defocus_angle_degrees,
//                          int image_width, int image_height);
//
//   vec3 random_in_unit_disk();                                    // x,y in the disk, z == 0
//
// Add tests_camera.cpp to CMakeLists.txt and call camera_tests() from tests().
//

#include "vec3.h"
#include "sphere.h"
#include "camera.h"      // or wherever make_frame / make_viewport live
#include <cassert>
#include <cmath>
#include <iostream>

static bool near(double a, double b, double eps = 1e-9) { return std::fabs(a - b) < eps; }
static bool vec_near(const vec3& a, const vec3& b, double eps = 1e-9) {
    return near(a.x(), b.x(), eps) && near(a.y(), b.y(), eps) && near(a.z(), b.z(), eps);
}

// ---------------------------------------------------------------- cross product
static void test_cross() {
    assert(vec_near(cross(vec3(1,0,0), vec3(0,1,0)), vec3(0,0,1)));      // right-handed: x cross
     y = z
    assert(vec_near(cross(vec3(0,1,0), vec3(0,0,1)), vec3(1,0,0)));      // y cross z = x
    assert(vec_near(cross(vec3(0,0,1), vec3(1,0,0)), vec3(0,1,0)));      // z cross x = y
    assert(vec_near(cross(vec3(0,1,0), vec3(1,0,0)), vec3(0,0,-1)));     // anticommutative
    assert(vec_near(cross(vec3(1,2,3), vec3(4,5,6)), vec3(-3,6,-3)));
    assert(vec_near(cross(vec3(2,3,4), vec3(2,3,4)), vec3(0,0,0)));      // parallel -> zero
    vec3 a(1,2,3), b(-2,0.5,4);
    vec3 c = cross(a, b);
    assert(near(dot(c, a), 0.0) && near(dot(c, b), 0.0));                // perpendicular to both
    // |a x b|^2 == |a|^2 |b|^2 - (a.b)^2   (Lagrange identity)
    assert(near(c.length_squared(), a.length_squared()*b.length_squared() - dot(a,b)*dot(a,b)));
}

// ---------------------------------------------------------------- camera frame
static void test_frame_orthonormal_right_handed() {
    camera_frame f = make_frame(vec3(-2,2,1), vec3(0,0,-1), vec3(0,1,0));
    assert(near(f.u.length(), 1.0) && near(f.v.length(), 1.0) && near(f.w.length(), 1.0));
    assert(near(dot(f.u, f.v), 0.0) && near(dot(f.v, f.w), 0.0) && near(dot(f.w, f.u), 0.0));
    assert(vec_near(cross(f.u, f.v), f.w));                              // right-handed
}

static void test_frame_directions() {
    camera_frame f = make_frame(vec3(-2,2,1), vec3(0,0,-1), vec3(0,1,0));
    // w points from lookat back toward lookfrom; the gaze is -w
    assert(dot(f.w, vec3(-2,2,1) - vec3(0,0,-1)) > 0.0);
    assert(vec_near(f.w, unit_vector(vec3(-2,2,2))));
    // with world-up as vup, u is horizontal and v leans up
    assert(near(f.u.y(), 0.0));
    assert(f.v.y() > 0.0);
}

static void test_frame_reduces_to_fixed_camera() {
    // the original hard-coded camera: eye at origin looking down -z, y up
    camera_frame f = make_frame(vec3(0,0,0), vec3(0,0,-1), vec3(0,1,0));
    assert(vec_near(f.u, vec3(1,0,0)));
    assert(vec_near(f.v, vec3(0,1,0)));
    assert(vec_near(f.w, vec3(0,0,1)));
}

static void test_frame_vup_need_not_be_perpendicular() {
```

```
    // vup with a component along the gaze: v must still come out perpendicular to w
    camera_frame f = make_frame(vec3(0,0,0), vec3(0,0,-1), vec3(0.3,1,-0.7));
    assert(near(dot(f.v, f.w), 0.0));
    assert(near(f.v.length(), 1.0));
    assert(vec_near(f.w, vec3(0,0,1)));
}

// ------------------------------------------------------------------ field of view
static void test_fov() {
    assert(near(viewport_height_from_fov(90.0, 1.0), 2.0));              // recovers the book's
    2.0
    assert(near(viewport_height_from_fov(90.0, 3.0), 6.0));              // scales with focal
    length
    assert(near(viewport_height_from_fov(2.0*std::atan(0.5)*180.0/M_PI, 1.0), 1.0));  // half
    height
    assert(viewport_height_from_fov(30.0, 1.0) < viewport_height_from_fov(60.0, 1.0)); //
    narrower = zoom
}

// ------------------------------------------------------------------ viewport
static void test_viewport_matches_fixed_camera() {
    // The general camera at the original pose must reproduce the numbers you printed on day 1.
    viewport vp = make_viewport(vec3(0,0,0), vec3(0,0,-1), vec3(0,1,0), 90.0, 0.0, 400, 225);
    assert(vec_near(vp.pixel00_loc,   vec3(-1.7733333333, 0.9955555556, -1.0), 1e-6));
    assert(vec_near(vp.pixel_delta_u, vec3(0.0088888889, 0.0, 0.0), 1e-9));
    assert(vec_near(vp.pixel_delta_v, vec3(0.0, -0.0088888889, 0.0), 1e-9));
    assert(vec_near(vp.center, vec3(0,0,0)));
    assert(vec_near(vp.defocus_disk_u, vec3(0,0,0)) && vec_near(vp.defocus_disk_v, vec3(0,0,0)));
}

static void test_viewport_center_is_lookat() {
    // The viewport plane passes through lookat (focal length = |lookfrom - lookat|),
    // so the pixel at the image center sits at lookat itself.
    const int W = 400, H = 225;
    vec3 lookfrom(-2,2,1), lookat(0,0,-1);
    viewport vp = make_viewport(lookfrom, lookat, vec3(0,1,0), 60.0, 0.0, W, H);
    vec3 center_px = vp.pixel00_loc + (W/2.0 - 0.5) * vp.pixel_delta_u + (H/2.0 - 0.5) * vp.
    pixel_delta_v;
    assert(vec_near(center_px, lookat, 1e-9));
    assert(vec_near(vp.center, lookfrom));
}

static void test_viewport_deltas_follow_frame() {
    camera_frame f = make_frame(vec3(-2,2,1), vec3(0,0,-1), vec3(0,1,0));
    viewport vp = make_viewport(vec3(-2,2,1), vec3(0,0,-1), vec3(0,1,0), 60.0, 0.0, 400, 225);
    // pixel steps point along +u and -v, and are square
    assert(vec_near(unit_vector(vp.pixel_delta_u), f.u));
    assert(vec_near(unit_vector(vp.pixel_delta_v), -1.0 * f.v));
    assert(near(vp.pixel_delta_u.length(), vp.pixel_delta_v.length()));
    // every pixel lies in the plane through lookat perpendicular to w
    vec3 far_px = vp.pixel00_loc + 399 * vp.pixel_delta_u + 224 * vp.pixel_delta_v;
    assert(near(dot(far_px - vec3(0,0,-1), f.w), 0.0));
}

static void test_zoom_shrinks_pixel_step() {
    viewport wide   = make_viewport(vec3(-2,2,1), vec3(0,0,-1), vec3(0,1,0), 90.0, 0.0, 400, 225)
    ;
    viewport narrow = make_viewport(vec3(-2,2,1), vec3(0,0,-1), vec3(0,1,0), 20.0, 0.0, 400, 225)
    ;
    assert(narrow.pixel_delta_u.length() < wide.pixel_delta_u.length());
    // ratio of pixel steps == ratio of tan(fov/2), not of the angles
    double expected = std::tan(10.0*M_PI/180.0) / std::tan(45.0*M_PI/180.0);
    assert(near(narrow.pixel_delta_u.length() / wide.pixel_delta_u.length(), expected, 1e-9));
}
```

```
// ------------------------------------------------------------------ sphere hits from a MOVED
    camera
// hit_sphere was only ever exercised with the camera at the origin. The positionable camera
// changes that, so these pin down the geometry for an arbitrary origin.
static void test_hit_sphere_from_moved_camera() {
    sphere s{vec3(0,0,-1), 0.5};
    // straight on from the origin, unit direction: near root at t = 0.5, hit point on the
    surface
    double t0 = hit_sphere(vec3(0,0,-1), vec3(0,0,0), s);
    assert(near(t0, 0.5));
    // same ray, direction doubled: t halves (t is in units of the direction vector)
    assert(near(hit_sphere(vec3(0,0,-2), vec3(0,0,0), s), 0.25));
    // camera moved to (0,0,+1), still looking down -z: sphere is now 2 units away, near root t =
     1.5
    assert(near(hit_sphere(vec3(0,0,-1), vec3(0,0,1), s), 1.5));
    // camera moved sideways to (3,0,-1), looking along -x at the sphere: near root t = 2.5
    assert(near(hit_sphere(vec3(-1,0,0), vec3(3,0,-1), s), 2.5));
    // the hit point returned by any of these lies on the sphere surface
    vec3 o(3,0,-1), dir(-1,0,0);
    double t = hit_sphere(dir, o, s);
    assert(near((o + t*dir - s.center).length(), s.radius));
    // a clear miss from a moved camera reports no hit (negative sentinel)
    assert(hit_sphere(vec3(0,1,0), vec3(3,0,-1), s) < 0.0);
}

// ------------------------------------------------------------------ defocus blur
static void test_random_in_unit_disk() {
    for (int i = 0; i < 2000; i++) {
        vec3 p = random_in_unit_disk();
        assert(p.x()*p.x() + p.y()*p.y() < 1.0);
        assert(near(p.z(), 0.0));
    }
}

static void test_defocus_disk_geometry() {
    vec3 lookfrom(-2,2,1), lookat(0,0,-1);
    double focus_dist = (lookfrom - lookat).length();
    viewport vp = make_viewport(lookfrom, lookat, vec3(0,1,0), 60.0, 10.0, 400, 225);
    camera_frame f = make_frame(lookfrom, lookat, vec3(0,1,0));
    double expected_radius = focus_dist * std::tan(5.0*M_PI/180.0);
    assert(near(vp.defocus_disk_u.length(), expected_radius, 1e-9));
    assert(near(vp.defocus_disk_v.length(), expected_radius, 1e-9));
    assert(vec_near(unit_vector(vp.defocus_disk_u), f.u));        // disk spans the u-v plane
    assert(vec_near(unit_vector(vp.defocus_disk_v), f.v));
    assert(near(dot(vp.defocus_disk_u, f.w), 0.0));
}

static void test_zero_defocus_angle_means_no_blur() {
    viewport vp = make_viewport(vec3(-2,2,1), vec3(0,0,-1), vec3(0,1,0), 60.0, 0.0, 400, 225);
    assert(near(vp.defocus_disk_u.length(), 0.0) && near(vp.defocus_disk_v.length(), 0.0));
}

void camera_tests() {
    test_cross();
    test_frame_orthonormal_right_handed();
    test_frame_directions();
    test_frame_reduces_to_fixed_camera();
    test_frame_vup_need_not_be_perpendicular();
    test_fov();
    test_viewport_matches_fixed_camera();
    test_viewport_center_is_lookat();
    test_viewport_deltas_follow_frame();
    test_zoom_shrinks_pixel_step();
```

```
    test_hit_sphere_from_moved_camera();
    test_random_in_unit_disk();
    test_defocus_disk_geometry();
    test_zero_defocus_angle_means_no_blur();
    std::cout << "camera tests passed\n";
}
```

Appendix: what was cut, and why

Antialiasing (§8) averages many rays per pixel. It is orthogonal to camera geometry, and returns only as an optional extension to defocus blur. Diffuse and metal materials, scattering, recursion depth, shadow acne, gamma correction (§9–§11): what light does after it leaves the camera. The `hittable` abstraction, `hit_record`, `hittable_list`, the `interval` class, `shared_ptr`, and the camera class refactor (§6.3–§7): software architecture that a one-geometry project does not need, and that would displace the C-style discipline being practiced. Front-face determination (§6.4): only meaningful for materials with two sides. Dielectrics (§11): materials again. The book's final scene (§14): a showcase for materials. Everything kept is a prerequisite for the camera model or a direct test of it.